UPF
UNIVERSIDADE DE PASSO FUNDO

MINICURSO · SENAINFO 2026

MicroPython na ESP32

Apostila prática: do primeiro LED ao servidor web, com Python rodando direto no microcontrolador.

Matheus Sebastiani Silva
Gustavo Luiz Rosset
Brenda Slongo Taca
Engenharia de Computação · Universidade de Passo Fundo



Sumário

	Apresentação	3
01	Conceitos	4
02	Thonny e o REPL	6
03	Saídas digitais	12
04	Entradas digitais	15
05	PWM: brilho e som	17
06	Potenciômetro e toque	19
07	Timers	21
08	Wi-Fi e servidor web	22
09	Rodando sem o computador	24
10	Desafio final	25
A	Cola rápida	26
B	Soluções	27
C	Glossário	29

Apresentação

Esta apostila acompanha o minicurso **MicroPython na ESP32**, uma aula prática de até 2h30. Cada participante recebe uma ESP32 **já gravada com MicroPython** e, ao final, terá controlado LEDs, lido sensores e comandado a placa pelo celular. Todos os códigos estão na pasta `exemplos/`.

> **Para quem é**
>
> Estudantes do 1º ao 3º semestre de Engenharia e Ciência da Computação. Basta saber o básico de programação (variável, `if`, laço e função). **Não é preciso saber eletrônica**: o pouco que usamos está explicado no capítulo 3. Termos novos estão no **Glossário** (Apêndice C).

▲ Cronograma

Horário	Atividade
19:00	Conceitos: microcontrolador, ESP32 e MicroPython
19:15	Thonny, REPL e o essencial de Python
19:40	LED, semáforo e botão
20:05	*Intervalo*
20:15	PWM, buzzer, potenciômetro e toque
20:40	Timers
20:55	Wi-Fi e servidor web
21:15	`main.py` e desafio final

▲ Material por dupla

ESP32 DevKit V1 com MicroPython, cabo micro-USB **de dados**, protoboard, jumpers, 3 LEDs (vermelho, amarelo, verde), 3 resistores de 220 Ω, 1 botão, 1 potenciômetro de 10 kΩ, 1 buzzer passivo e um computador com o **Thonny**.

01
Conceitos

Microcontrolador

Um **computador pequeno em um único chip**, feito para controlar coisas: ler sensores, decidir e acionar LEDs, motores e afins. Comparado ao seu notebook, tem bem menos memória (kB em vez de GB), roda um programa só e, em vez de tela e teclado, conversa com o mundo por **pinos**. Está em micro-ondas, carros, controles remotos e na Internet das Coisas (IoT).

ESP32

Microcontrolador barato e muito popular, com **Wi-Fi e Bluetooth**. Seus pinos se chamam **GPIO** (pinos de uso geral): o programa pode ligar, desligar ou ler cada um deles. Eles trabalham com **3,3 V**.

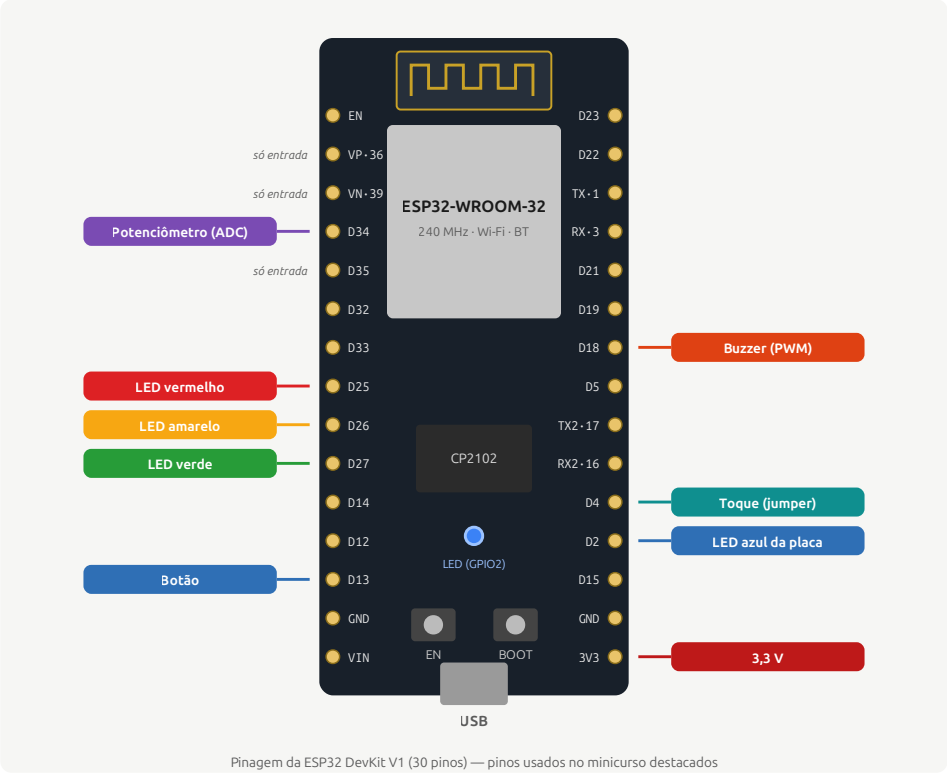


Figura 1. Pinagem da ESP32 DevKit V1, com os pinos usados no minicurso destacados.

MicroPython

É o **Python rodando dentro da placa**. Ele já foi instalado na ESP32 antes da aula; você só escreve arquivos `.py`. A grande vantagem: **escreveu, rodou, viu o resultado** — sem compilar e gravar de novo a cada mudança.

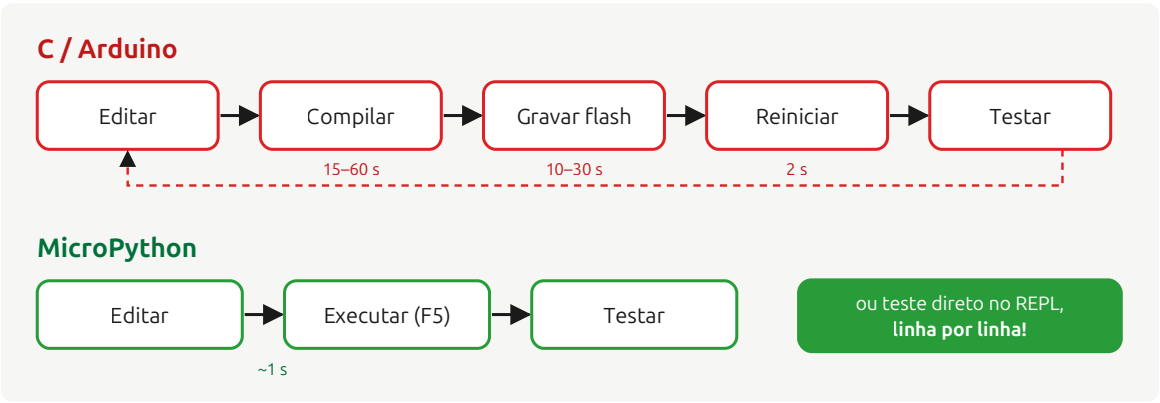


Figura 2. Em C é preciso compilar e regravar a cada mudança; no MicroPython basta executar.

Vantagens

- Fácil de aprender (é Python)
- Testa na hora, pelo REPL
- Código curto

Desvantagens

- Mais lento que C
- Usa mais memória

02
Thonny e o REPL

▲ Instalando o Thonny

O **Thonny** é um editor de Python gratuito, pensado para quem está começando. Ele já sabe conversar com placas que rodam MicroPython: envia o código pelo cabo USB e mostra a resposta da placa.

- **Windows e macOS:** baixe o instalador em **thonny.org**. Instale também o driver do chip USB da placa: **CP210x** (Silicon Labs) ou **CH340** (WCH). Olhe o chip ao lado do conector USB para saber qual é.
- **Linux:** `sudo apt install thonny` ou `pip install thonny`. O driver já vem no sistema. Para usar a porta serial sem `sudo`, adicione seu usuário ao grupo `dialout` (`sudo usermod -aG dialout $USER`) e faça logout.

Ao abrir pela primeira vez, o Thonny roda o código no **Python do próprio computador**. O canto inferior direito mostra **Local Python 3**, e o Shell mostra a versão do Python do PC.

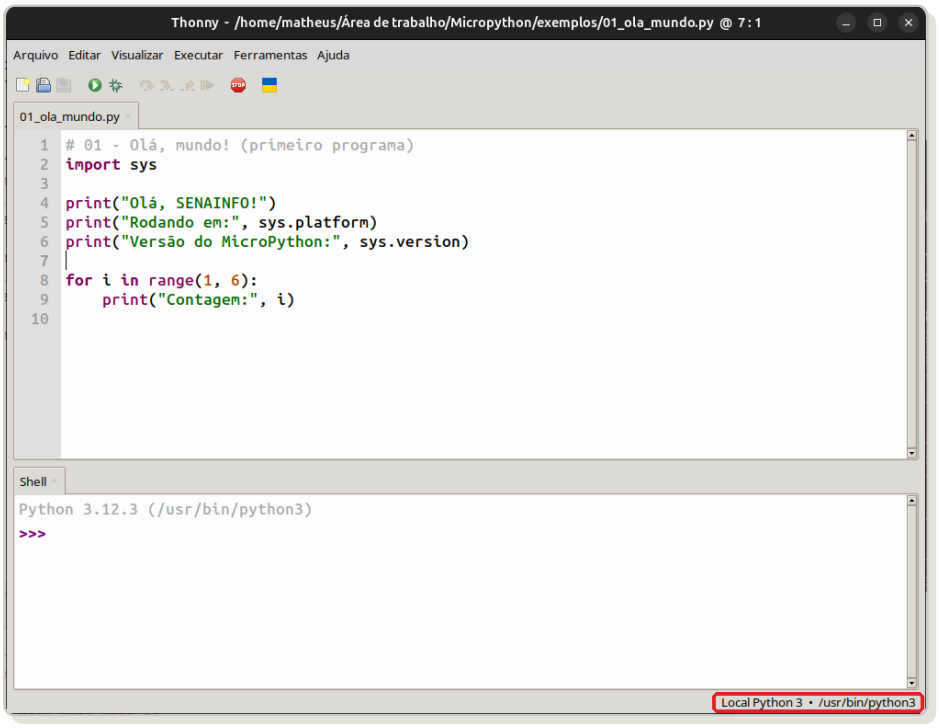


Figura 3. Thonny recém-instalado: o interpretador ainda é o Python do computador (destaque em vermelho).

▲ Configurando o interpretador para o MicroPython

O **interpretador** define **onde** o código roda. Precisamos trocar o Python do computador pelo MicroPython da ESP32. Isso é feito uma única vez, e o Thonny guarda a escolha.

1. Conecte a ESP32 no USB **antes** de abrir a configuração. A lista de portas só é lida quando a janela abre.

2. No menu, clique em **Executar → Configurar interpretador…**.

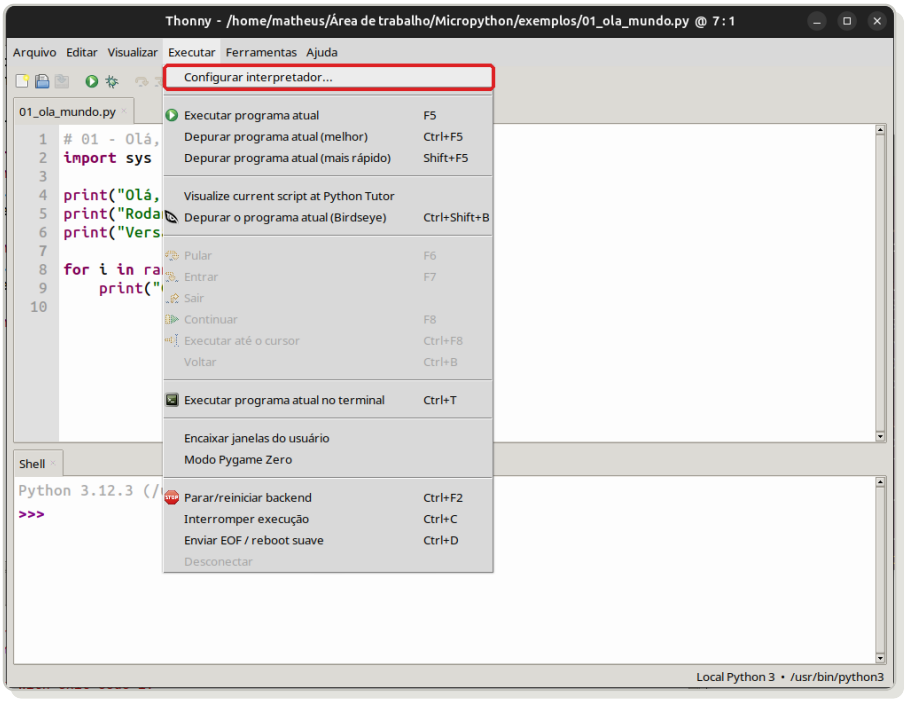


Figura 4. Menu Executar → Configurar interpretador…

3. Na janela **Opções do Thonny**, aba **Interpretador**, abra a primeira lista e escolha **MicroPython (ESP32)**.

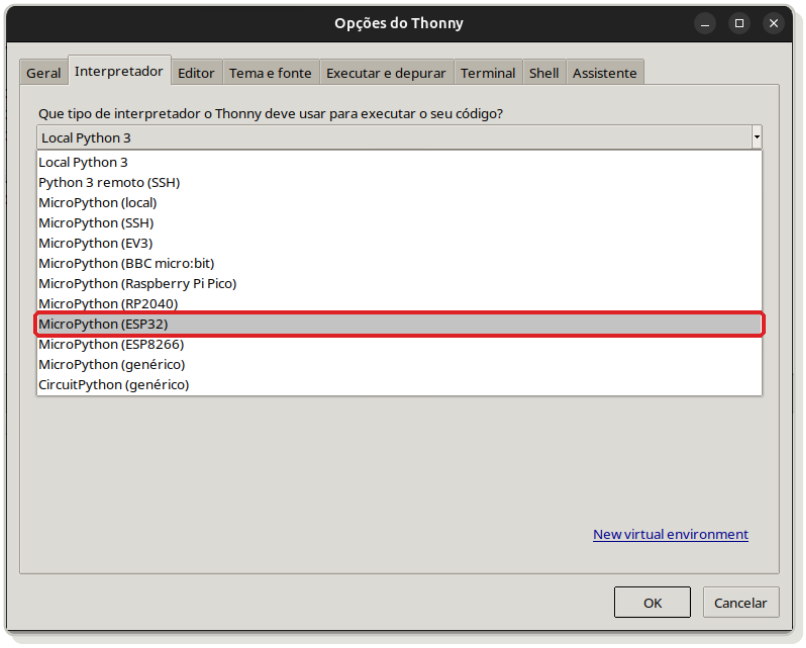


Figura 5. Escolhendo o tipo de interpretador: MicroPython (ESP32).

4. Em **Porta ou WebREPL**, escolha a porta da placa. Ela aparece com o nome do chip USB: **CP2102 USB to UART Bridge Controller**, **CH340** ou **USB Serial**. No Linux a porta é `/dev/ttyUSB0`; no Windows, `COM3`, `COM4` …; no macOS, `/dev/cu.usbserial-…`.

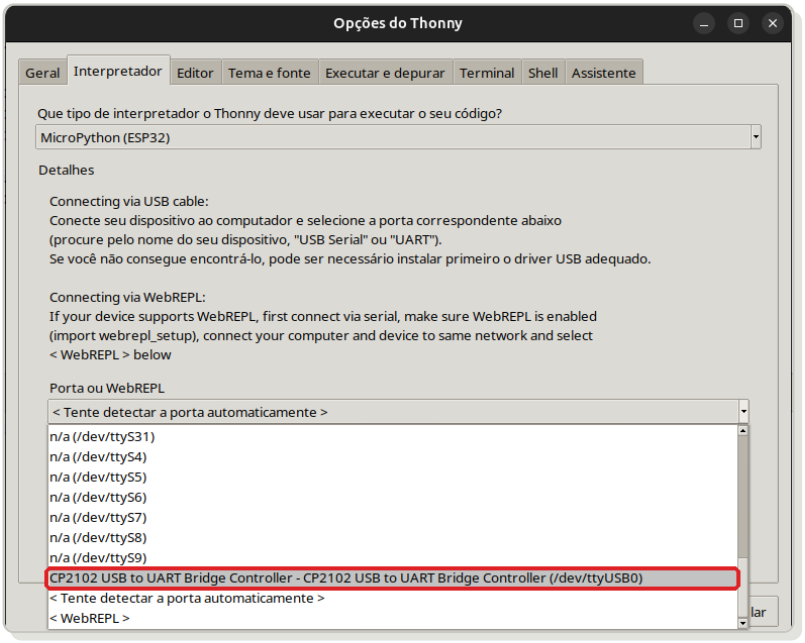


Figura 6. Escolhendo a porta: a ESP32 aparece como CP2102 (/dev/ttyUSB0).

5. Confira as duas escolhas e clique em **OK**.

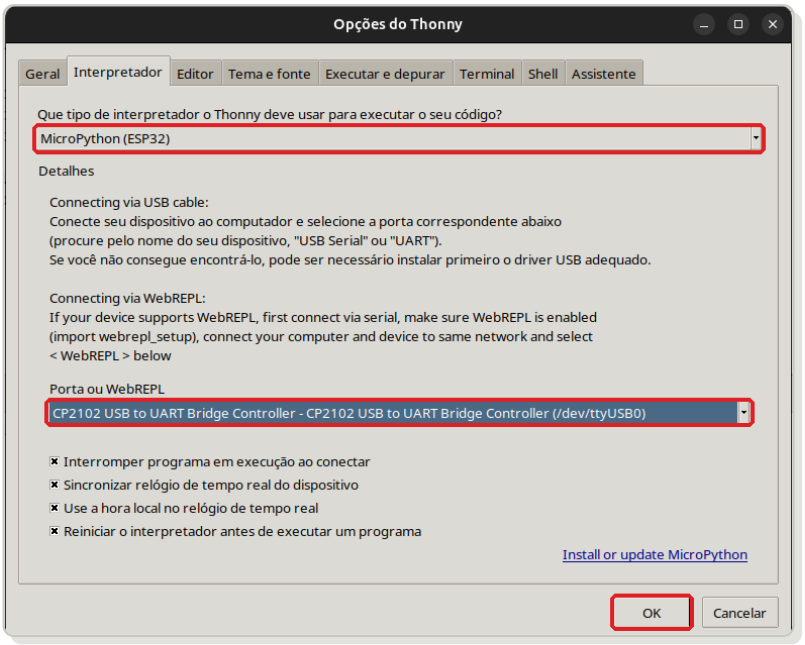


Figura 7. Configuração pronta: MicroPython (ESP32) na porta da placa.

> **Não sabe qual é a porta?**
>
> Desconecte a placa e abra a lista; conecte de novo e reabra a janela. A porta que apareceu é a da ESP32. A opção **< Tente detectar a porta automaticamente >** também costuma funcionar.

Se deu certo, o Shell mostra `MicroPython v1.xx … with ESP32` e o prompt `>>>`, e o canto inferior direito passa a mostrar **MicroPython (ESP32) • /dev/ttyUSB0** (ou a sua porta).

▲ Não conectou?

Os dois erros mais comuns aparecem em vermelho no Shell:

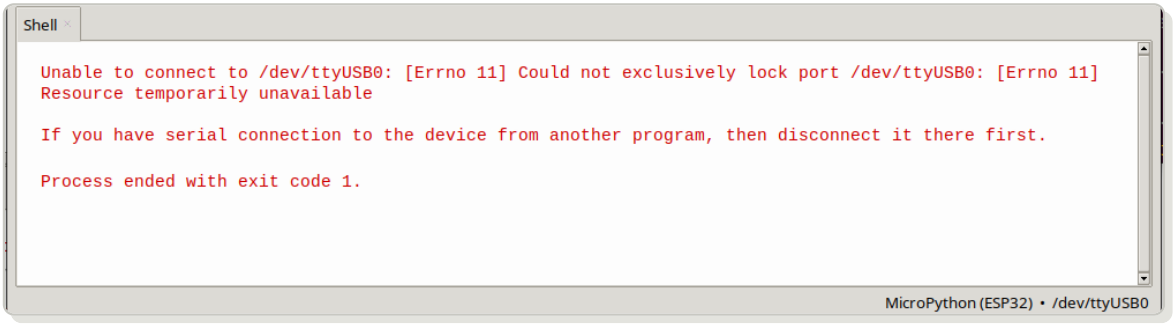


Figura 8. *Could not exclusively lock port*: outro programa está usando a porta.



Figura 9. *No such file or directory*: o computador não encontrou a placa.

Como resolver

- **Porta ocupada:** feche o outro programa que usa a placa (outra janela do Thonny, Arduino IDE, monitor serial) ou use **Executar → Desconectar** nele.
- **Placa não encontrada:** confira o cabo (cabos só de carga não transmitem dados), troque de porta USB e verifique o driver.
- Depois de resolver, clique em **STOP** (Ctrl+F2) para o Thonny tentar conectar de novo.

▲ A janela do Thonny

Em cima fica o **editor**; embaixo, o **Shell**, que mostra a saída e o prompt. Use **F5** (botão verde) para executar e **STOP** (Ctrl+F2) para parar.

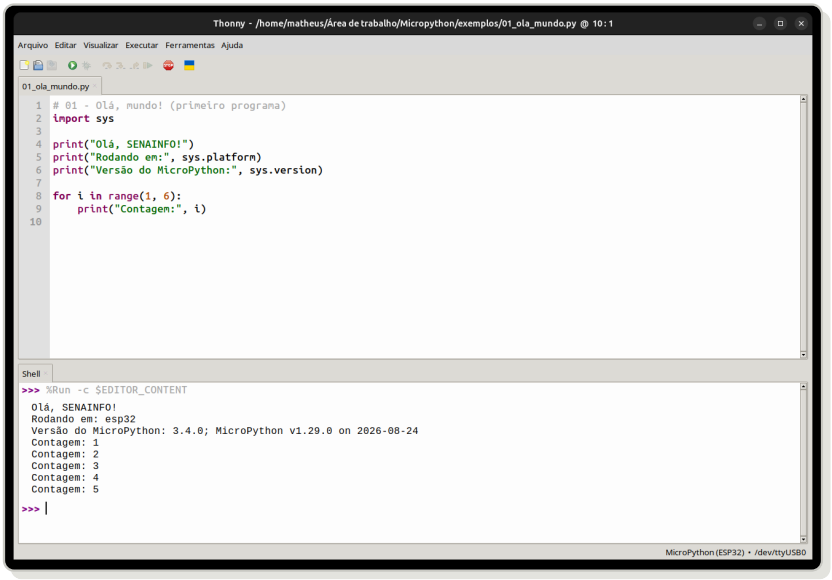


Figura 10. Programa 01_ola_mundo.py executado na ESP32.

▲ REPL

O REPL (o prompt `>>>`) executa cada linha assim que você aperta Enter. Experimente no Shell — ao rodar `led.on()`, o LED azul da placa acende:

```
from machine import Pin
led = Pin(2, Pin.OUT)
led.on()
led.off()
```

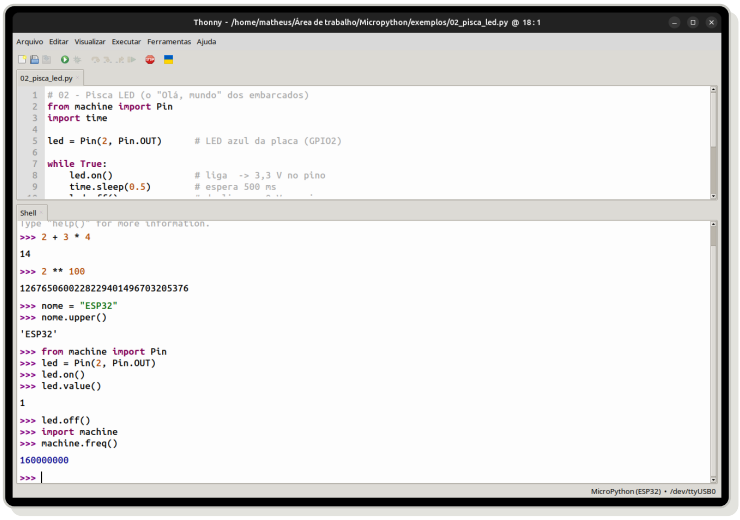


Figura 11. Comandos digitados no REPL.

▲ Python essencial

```
nome = "ESP32"            # variável
leds = [25, 26, 27]       # lista

for p in leds:            # laço
    print("pino", p)

if len(leds) > 2:         # condição
    print("três LEDs")

def dobro(x):             # função
    return 2 * x
```

Quem vem do C: aqui não há chaves nem ponto e vírgula — a **indentação** (4 espaços) define o que está dentro do `if`, do laço e da função.

> **Deu erro?**
>
> Leia a **última linha** da mensagem: ela diz o tipo do erro. A linha com `line N` diz onde. Os mais comuns: `NameError` (nome escrito errado), `IndentationError` (espaços errados) e `ValueError` (valor que não serve, como um pino inexistente).

03
Saídas digitais

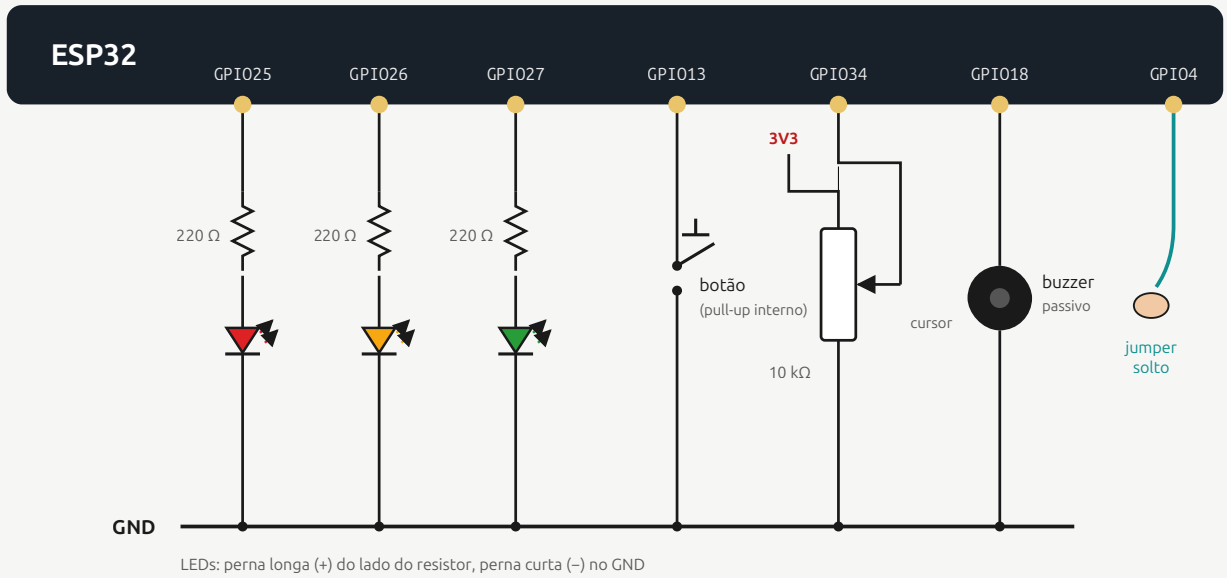
▲ Eletrônica em 1 minuto

- **Digital:** pino ligado = 3,3 V = **1**; pino desligado = 0 V = **0**. O **GND** é o 0 V (o “negativo”).
- **LED:** só acende num sentido. A **perna longa** vai para o lado do pino; a **curta**, para o GND.
- **Resistor:** limita a corrente para o LED não queimar. Usamos 220 Ω, e ele não tem lado certo.

Pense em água num cano: a tensão é a pressão, a corrente é o fluxo e o resistor é um trecho de cano mais fino.

▲ Montagem

Monte o circuito com a placa **desligada** do USB. Ele será usado no resto da aula.



Esquema de montagem do kit (todos os componentes compartilham o GND da placa)

Figura 12. Esquema do kit: LEDs nos GPIOs 25, 26 e 27 (com 220 Ω), botão no 13, potenciômetro no 34, buzzer no 18 e jumper de toque no 4.

▲ Pisca LED

02_pisca_led.py

```python
# 02 - Pisca LED (o "Olá, mundo" dos embarcados)
from machine import Pin
import time

led = Pin(2, Pin.OUT)      # LED azul da placa (GPIO2)

while True:
    led.on()               # liga  -> 3,3 V no pino
    time.sleep(0.5)        # espera 500 ms
    led.off()              # desliga -> 0 V no pino
    time.sleep(0.5)
```

`Pin(2, Pin.OUT)` define o pino como saída; `on()` e `off()` ligam e desligam; `sleep(0.5)` espera meio segundo; `while True` repete para sempre.

> 🎯 **Desafio**
>
> Faça o LED vermelho (GPIO25) piscar um “SOS”: 3 piscadas curtas, 3 longas e 3 curtas. *Solução no Apêndice B.*

▲ Semáforo

10_semaforo.py

```python
# 10 - Semáforo com três LEDs e botão de pedestre
from machine import Pin
import time

vermelho = Pin(25, Pin.OUT)
amarelo  = Pin(26, Pin.OUT)
verde    = Pin(27, Pin.OUT)
botao    = Pin(13, Pin.IN, Pin.PULL_UP)

def acende(cor):
    for led in (vermelho, amarelo, verde):
        led.off()
    cor.on()

def espera(segundos):
    # espera em passos de 10 ms, olhando o botão a cada passo
    for i in range(segundos * 100):
        if botao.value() == 0:
            print("Pedestre pediu passagem!")
            return
        time.sleep_ms(10)

while True:
    acende(verde)
    espera(5)      # ou menos, se apertarem
    acende(amarelo)
    time.sleep(2)
    acende(vermelho)
    time.sleep(4)
```

`acende()` apaga os três LEDs e acende só o escolhido. Já `espera()` não usa um único `sleep(5)`, porque durante o `sleep` a placa não faz mais nada e não veria o botão. Em vez disso, ela espera em 500 passos de 10 ms e **olha o botão a cada passo**: se o pedestre apertar, o sinal muda antes.

04
Entradas digitais

Lendo um botão

O botão liga o pino ao GND. Por isso a leitura parece “invertida”: **1** com o botão solto e **0** quando apertado. O `Pin.PULL_UP` liga um resistor interno que segura o pino em 1 enquanto ninguém aperta; sem ele, o pino fica “indeciso” e lê valores aleatórios.

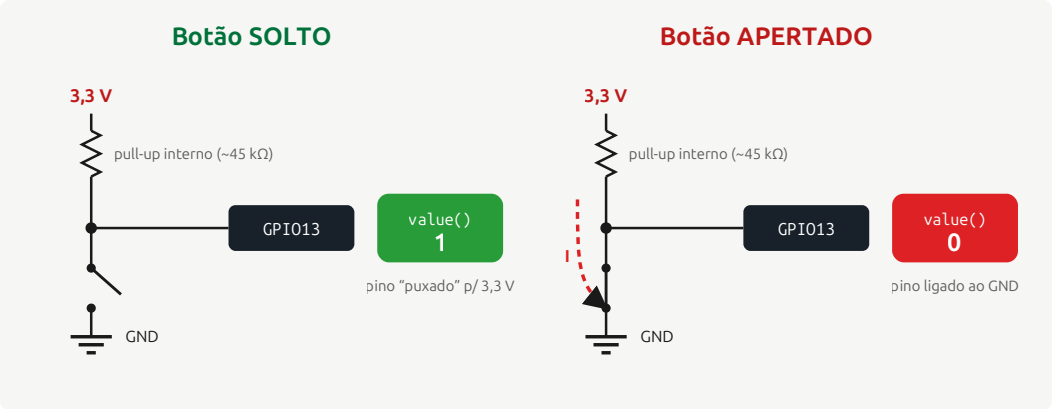


Figura 13. Botão com pull-up interno.

03_botao.py

```python
# 03 - Lendo um botão (entrada digital com pull-up interno)
from machine import Pin
import time

led   = Pin(2, Pin.OUT)
botao = Pin(13, Pin.IN, Pin.PULL_UP)   # solto = 1, apertado = 0

while True:
    if botao.value() == 0:             # apertado (ligado ao GND)
        led.on()
    else:
        led.off()
    time.sleep_ms(20)                  # pequeno atraso (debounce simples)
```

Interrupções

Ficar perguntando ao botão o tempo todo é como ficar olhando pela janela esperando visita. A **interrupção** é a campainha: o hardware avisa e a função `ao_apertar()` é chamada **sozinha** quando o botão é apertado. Mantenha essa função curta.

- `global` deixa a função alterar variáveis criadas fora dela.

- Botões mecânicos “tremem” e geram vários pulsos num só aperto. O `if` dos 200 ms ignora esses pulsos extras.

04_interrupcao.py

```python
# 04 - Interrupção: o hardware avisa quando o botão é pressionado
from machine import Pin
import time

led   = Pin(2, Pin.OUT)
botao = Pin(13, Pin.IN, Pin.PULL_UP)
contador = 0
ultimo = 0

def ao_apertar(pino):
    global contador, ultimo
    agora = time.ticks_ms()
    if time.ticks_diff(agora, ultimo) > 200:   # ignora repiques (debounce)
        contador += 1
        led.value(not led.value())             # inverte o LED
        ultimo = agora

botao.irq(trigger=Pin.IRQ_FALLING, handler=ao_apertar)

while True:                                    # o laço principal fica livre!
    print("Apertos:", contador)
    time.sleep(1)
```

05
PWM: brilho e som

Um pino digital só sabe ligar ou desligar. Para ter “meio brilho”, o PWM liga e desliga o pino tão rápido que o olho enxerga a média: quanto mais tempo ligado (o *duty*), mais brilho. `duty_u16(0)` é apagado e `duty_u16(65535)` é o brilho máximo (65 535 é o maior número de 16 bits).

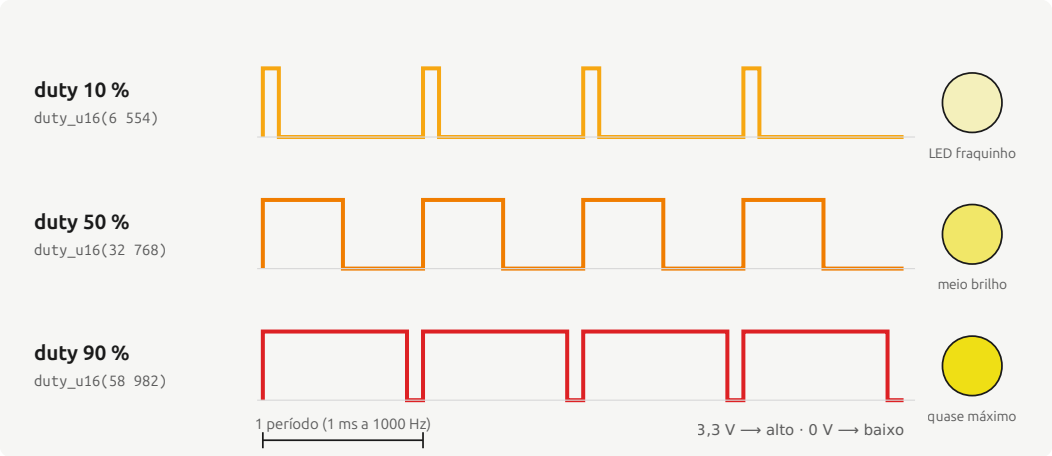


Figura 14. Sinais PWM com duty de 10 %, 50 % e 90 %.

05_pwm_fade.py

```python
# 05 - PWM: controlando o brilho do LED
from machine import Pin, PWM
import time

led = PWM(Pin(2), freq=1000)        # 1000 Hz

while True:
    for brilho in range(0, 65536, 1024):     # sobe: 0 % -> 100 %
        led.duty_u16(brilho)
        time.sleep_ms(15)
    for brilho in range(65535, -1, -1024):   # desce: 100 % -> 0 %
        led.duty_u16(brilho)
        time.sleep_ms(15)
```

▲ Buzzer

No buzzer, a **frequência** (quantas vezes por segundo o pino liga e desliga) define a nota: 440 Hz é o Lá. As notas ficam num dicionário. Experimente mudar a `musica`.

06_buzzer.py

```python
# 06 - Tocando notas em um buzzer passivo com PWM
from machine import Pin, PWM
import time

buzzer = PWM(Pin(18), freq=440, duty_u16=0)

NOTAS = {"C": 262, "D": 294, "E": 330, "F": 349,
         "G": 392, "A": 440, "B": 494, "-": 0}

musica = "C D E F - F F - C D C D - D D - C G F E - E E - C D E F - F F"

def tocar(nota, duracao_ms=200):
    if NOTAS[nota] == 0:
        buzzer.duty_u16(0)              # pausa = silêncio
    else:
        buzzer.freq(NOTAS[nota])
        buzzer.duty_u16(32768)          # 50 % de duty -> onda quadrada
    time.sleep_ms(duracao_ms)
    buzzer.duty_u16(0)
    time.sleep_ms(30)

for n in musica.split():
    tocar(n)
buzzer.deinit()
print("Fim da música!")
```

06
Potenciômetro e toque

▲ Leitura analógica (ADC)

O potenciômetro é um botão de girar que muda a tensão no pino entre 0 e 3,3 V. O **ADC** transforma essa tensão em um número: 0 V vira 0 e 3,3 V vira 65 535. Como o número já está na faixa do PWM, basta repassá-lo ao LED.

07_potenciometro.py

```
# 07 - ADC: lendo um potenciômetro e controlando o brilho
from machine import Pin, ADC, PWM
import time

pot = ADC(Pin(4))                  # GPIO4: entrada analógica (ADC2)
pot.atten(ADC.ATTN_11DB)           # lê de 0 a 3,3 V
led = PWM(Pin(2), freq=1000)

while True:
    valor = pot.read_u16()         # 0 ... 65535
    led.duty_u16(valor)            # brilho acompanha o potenciômetro
    print(valor)                   # o Plotter do Thonny desenha isso!
    time.sleep_ms(100)
```

▲ Plotter

Em **Visualizar → Plotter**, o Thonny desenha um gráfico com os números impressos pelo programa.

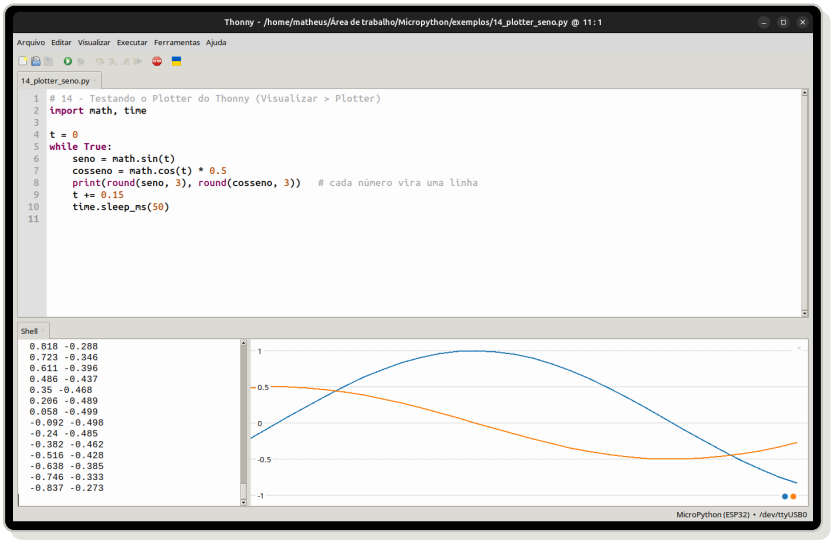


Figura 15. Plotter do Thonny com o exemplo 14_plotter_seno.py.

▲ Sensor de toque

Basta um jumper no GPIO4: ao encostar o dedo, o valor lido **diminui**. Ajuste o `LIMIAR` olhando o Plotter.

08_toque.py

```python
# 08 - Sensor de toque capacitivo (sem nenhum componente extra!)
from machine import Pin, TouchPad
import time

toque = TouchPad(Pin(4))      # basta um fio (jumper) no GPIO4
led = Pin(2, Pin.OUT)

LIMIAR = 300                  # ajuste olhando os valores no Plotter

while True:
    valor = toque.read()      # valor cai quando você encosta o dedo
    led.value(valor < LIMIAR)
    print(valor)
    time.sleep_ms(100)
```

07
Timers

Um timer é um despertador do hardware: a cada intervalo ele chama uma função, **sem travar** o programa. Com `sleep` seria complicado piscar dois LEDs em ritmos diferentes; com dois timers, cada um cuida do seu.

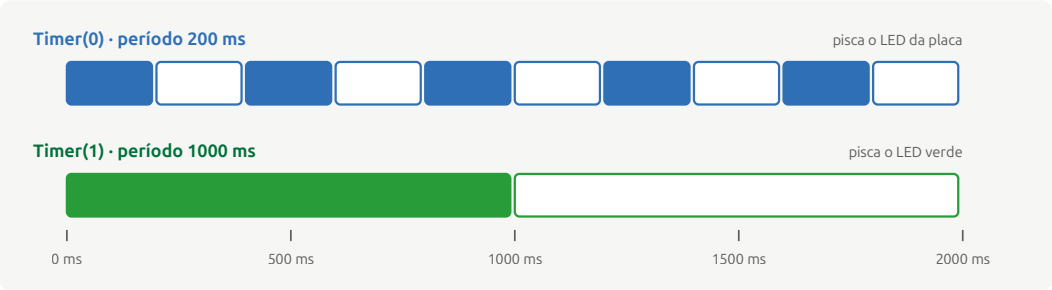


Figura 16. Dois timers independentes.

09_timer.py

```python
# 09 - Timers: várias tarefas "ao mesmo tempo", sem sleep()
from machine import Pin, Timer

led_placa = Pin(2, Pin.OUT)
led_verde = Pin(19, Pin.OUT)

def pisca_rapido(t):
    led_placa.value(not led_placa.value())

def pisca_lento(t):
    led_verde.value(not led_verde.value())

t0 = Timer(0)
t1 = Timer(1)
t0.init(period=200,  mode=Timer.PERIODIC, callback=pisca_rapido)
t1.init(period=1000, mode=Timer.PERIODIC, callback=pisca_lento)

print("Timers rodando! O REPL continua livre.")
print("Para parar: t0.deinit(); t1.deinit()")
```

Para parar, use `t0.deinit()` e `t1.deinit()`.

08
Wi-Fi e servidor web

▲ Procurando redes

12_wifi_scan.py

```python
# 12 - Procurando redes Wi-Fi
import network

wlan = network.WLAN(network.WLAN.IF_STA)   # modo cliente
wlan.active(True)

print("Procurando redes...")
redes = wlan.scan()          # cada rede é uma tupla com várias informações

for rede in redes:
    nome = rede[0].decode()  # posição 0: nome da rede
    sinal = rede[3]          # posição 3: força do sinal (dBm)
    print(sinal, "dBm", nome)
```

`scan()` devolve uma lista; cada rede é uma tupla, em que a posição 0 é o nome e a posição 3 é a força do sinal em dBm. Quanto mais perto de zero, mais forte (−40 é ótimo; −90 é fraco).

▲ Controlando o LED pelo celular

A ESP32 cria a **própria rede Wi-Fi**. O celular se conecta a ela e abre uma página com botões para ligar e desligar o LED.

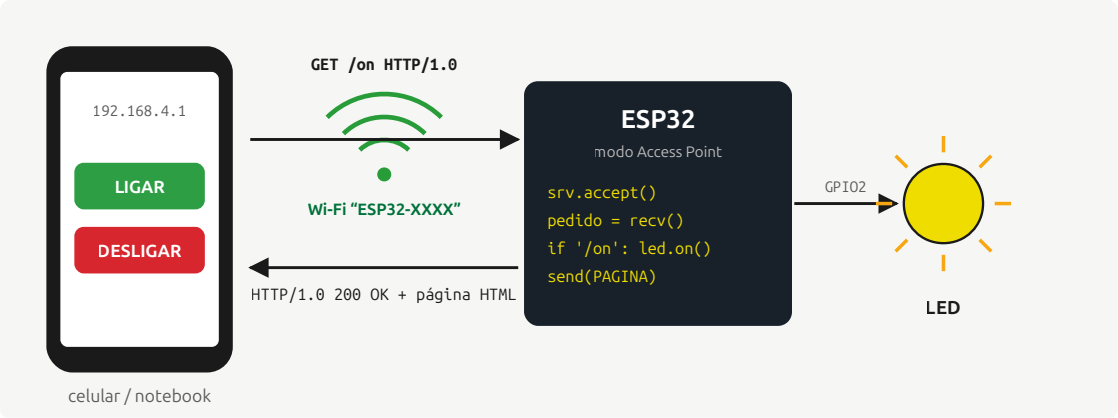


Figura 17. O navegador pede /on ou /off; a ESP32 aciona o LED e devolve a página.

✋ Passo a passo

1. Rode o `13_servidor_web.py` e veja o nome da rede no Shell (**ESP32-XXXX**).
2. No celular, conecte-se a ela (senha `senainfo`).
3. Abra **http://192.168.4.1** e toque em LIGAR / DESLIGAR.

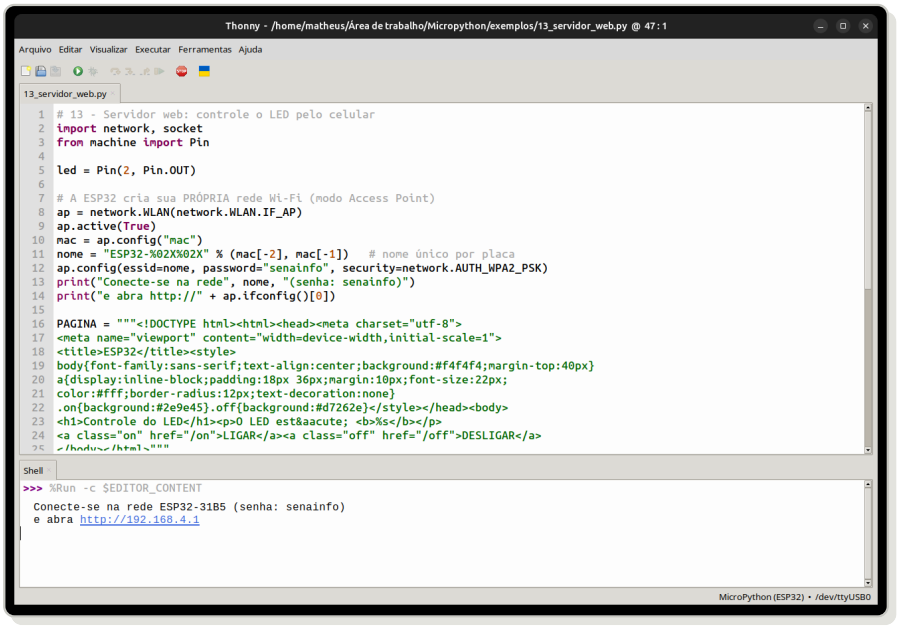


Figura 18. Servidor rodando no Thonny.



Figura 19. Página no celular.

O código completo está em `exemplos/13_servidor_web.py`. Não é preciso entender cada linha: a ideia é que o navegador pede um endereço (`/on` ou `/off`) e a placa responde com uma página HTML, que está no texto `PAGINA`. Experimente mudar o título e as cores dela.

09
Rodando sem o computador

Para a placa rodar o programa sozinha ao ser ligada, salve-o **na placa** com o nome `main.py`: **Arquivo → Salvar como → dispositivo MicroPython →** `main.py`.

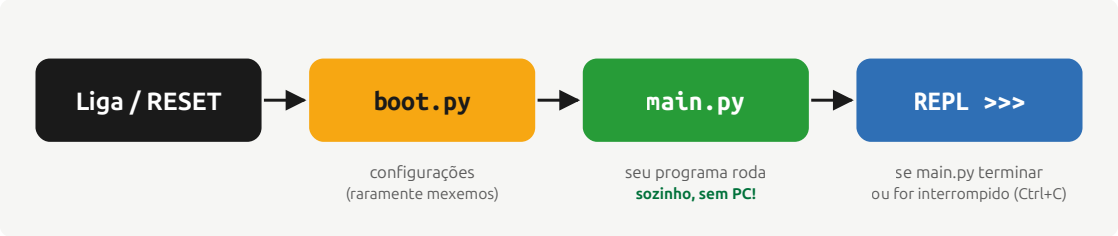


Figura 20. Ao ligar, a placa executa boot.py e depois main.py.

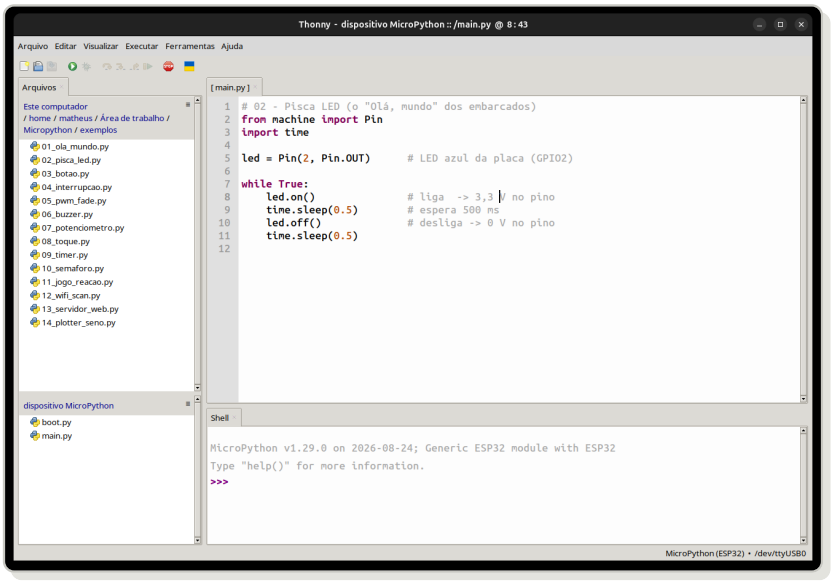


Figura 21. O main.py aparece na lista de arquivos da placa (Visualizar → Arquivos).

Para apagar, clique com o botão direito no `main.py` e escolha Excluir.

10
Desafio final

Jogo de reação: o LED acende num momento sorteado e o jogador aperta o botão o mais rápido possível. O programa mostra o tempo em milissegundos.

11_jogo_reacao.py

```
# 11 - Desafio: jogo de tempo de reação
from machine import Pin
import time, random

led   = Pin(2, Pin.OUT)
botao = Pin(13, Pin.IN, Pin.PULL_UP)
recorde = None

print("Aperte o botão assim que o LED acender!")

while True:
    led.off()
    time.sleep_ms(random.randint(1500, 5000))   # espera aleatória

    if botao.value() == 0:
        print("Queimou a largada!")
        time.sleep(2)
        continue

    led.on()
    inicio = time.ticks_ms()
    while botao.value() == 1:                   # espera o aperto
        pass
    reacao = time.ticks_diff(time.ticks_ms(), inicio)
    led.off()

    if recorde is None or reacao < recorde:
        recorde = reacao
        print("NOVO RECORDE:", reacao, "ms")
    else:
        print("Tempo:", reacao, "ms   (recorde:", recorde, "ms)")
    time.sleep(2)
```

🎯 Turbine o jogo

- Bip no buzzer quando o LED acender.
- LED verde quando houver novo recorde.
- Modo 2 jogadores.

Solução dos dois primeiros itens no Apêndice B.

A
Apêndice A — Cola rápida

resumo.py

```python
from machine import Pin, PWM, ADC
import time

led = Pin(2, Pin.OUT)                # saída
led.on()
led.off()

bt = Pin(13, Pin.IN, Pin.PULL_UP)    # entrada (0 = apertado)
bt.value()

pwm = PWM(Pin(25), freq=1000)        # PWM
pwm.duty_u16(32768)                  # 50 %

pot = ADC(Pin(34))                   # leitura analógica
pot.atten(ADC.ATTN_11DB)             # faixa 0 a 3,3 V
pot.read_u16()                       # 0 .. 65535

time.sleep(1)                        # 1 segundo
time.sleep_ms(10)                    # 10 milissegundos
```

Atalhos e problemas comuns

Situação	O que fazer
Executar / parar	**F5** / **Ctrl+F2** (STOP)
Placa não aparece	Troque o cabo (pode ser só de carga) ou instale o driver CP210x/CH340
Device is busy	Feche outros programas que usam a porta e aperte STOP
LED não acende	Confira a polaridade (perna longa no lado do resistor) e o GND
Placa travada	Aperte o botão **EN** e depois STOP

Próximos passos

- Novos sensores: temperatura (DHT22), distância (HC-SR04), telas pequenas (OLED).
- Projeto de disciplina: uma estação do tempo com página web.
- Documentação: **docs.micropython.org** · Simulador: **wokwi.com**

B
Apêndice B — Soluções

▲ Pisca “SOS”

solucoes/s1_sos.py

```python
# Solução - pisca "SOS" em código Morse no LED vermelho
from machine import Pin
import time

led = Pin(25, Pin.OUT)
PONTO, TRACO = 200, 600            # duração em ms

def sinal(duracao):
    led.on()
    time.sleep_ms(duracao)
    led.off()
    time.sleep_ms(200)             # intervalo entre sinais

while True:
    for d in (PONTO, PONTO, PONTO, TRACO, TRACO, TRACO, PONTO, PONTO, PONTO):
        sinal(d)
    time.sleep_ms(1500)            # pausa entre as palavras
```

▲ Jogo de reação turbinado

solucoes/s2_jogo_turbinado.py

```python
# Solução - jogo de reação com bip e LEDs de resultado
from machine import Pin, PWM
import time, random

led      = Pin(2, Pin.OUT)
verde    = Pin(27, Pin.OUT)
vermelho = Pin(25, Pin.OUT)
botao    = Pin(13, Pin.IN, Pin.PULL_UP)
buzzer   = PWM(Pin(18), freq=1000, duty_u16=0)
recorde  = None

def bip(freq=1000, ms=80):
    buzzer.freq(freq)
    buzzer.duty_u16(32768)
    time.sleep_ms(ms)
    buzzer.duty_u16(0)

while True:
    led.off(); verde.off(); vermelho.off()
    time.sleep_ms(random.randint(1500, 5000))
    if botao.value() == 0:
        print("Queimou a largada!")
        bip(200, 400)
        time.sleep(2)
        continue

    led.on()
    bip()                                  # aviso sonoro
    inicio = time.ticks_ms()
    while botao.value() == 1:
        pass
    reacao = time.ticks_diff(time.ticks_ms(), inicio)
    led.off()

    if recorde is None or reacao < recorde:
        recorde = reacao
        verde.on()
        bip(1500, 60); bip(2000, 60)       # som de vitória
        print("NOVO RECORDE:", reacao, "ms")
    else:
        vermelho.on()
        print("Tempo:", reacao, "ms  (recorde:", recorde, "ms)")
    time.sleep(2)
```

C

Apêndice C — Glossário

Termo	O que é
ADC	Conversor que transforma uma tensão (0 a 3,3 V) em um número.
Debounce	Ignorar os pulsos extras gerados quando um botão “treme” ao ser apertado.
dBm	Unidade da força do sinal Wi-Fi. Mais perto de zero = sinal mais forte.
Duty	Fração do tempo em que o sinal PWM fica ligado.
Firmware	Programa gravado na placa. Aqui, é o próprio MicroPython.
GND	O 0 V do circuito (o “negativo”).
GPIO	Pino de uso geral: o programa pode ligar, desligar ou ler.
Interrupção	Aviso do hardware que chama uma função na hora em que algo acontece.
IoT	Internet das Coisas: objetos ligados à rede.
Pull-up	Resistor que mantém um pino em 1 enquanto nada o puxa para 0.
PWM	Ligar e desligar um pino muito rápido para simular valores intermediários.
REPL	O prompt `>>>`: lê, executa e mostra cada linha na hora.
Timer	“Despertador” do hardware que chama uma função a cada intervalo.